

INFORME DE PRUEBAS DE SOFTWARE

basado en ISO 29119

Plantilla para cumplimentar por el alumno

Proyecto probado	Calculadora Básica
Pareja – Alumno 1	Pablo Bielsa
Pareja – Alumno 2	Nicolae Poenaru
Repositorio Git	https://github.com/bielsaa777/Calculadora
Fecha de inicio de pruebas	18/05/2026
Fecha de fin de pruebas	22/05/2026
Versión del documento	V1.0
Revisado por (profesor)	

PARTE 1 – Plan de Pruebas (ISO 29119)

El Plan de Pruebas describe QUÉ vais a probar, POR QUÉ, CÓMO y con qué recursos. Debe escribirse ANTES de empezar a probar.

1.1 Identificación y Alcance

Identificador del Plan	TP-001
Versión	1.0
Proyecto / Sistema bajo prueba (SUT)	Proyecto calculadora-basica, clase principal cesur.daml.practica.Calculadora.
Responsable de las pruebas	Pablo y Nico

1.2 Elementos a Probar

Listad los métodos/funcionalidades concretas que vais a probar. Sed específicos: no pongáis 'todo', poned 'Calculadora.dividir()', 'Calculadora.maximo()...'

Elementos bajo prueba

Calculadora.sumar(double a, double b) – Validación de operaciones de adición con valores positivos, negativos y neutros.

Calculadora.restar(double a, double b) – Verificación de sustracciones y control de desbordamientos de signos.

Calculadora.multiplicar(double a, double b) – Evaluación de productos y propiedad del elemento nulo (cero).

Calculadora.dividir(double a, double b) – Verificación de cocientes y robustez ante denominadores iguales a cero.

Calculadora.maximo(double a, double b) – Prueba del flujo lógico condicional para determinar el mayor de dos números.

Calculadora.porcentaje(double valor, double total) – Validación del cálculo de proporciones y control de divisiones por cero en el total

1.3 Estrategia de Pruebas

Explicad qué tipo de pruebas vais a hacer y por qué. Mencionad caja negra, caja blanca, valores límite...

Estrategia y enfoque de pruebas

Se aplica un nivel de Pruebas Unitarias automatizadas utilizando el framework JUnit 5. La estrategia combina técnicas de Caja Negra para asegurar el comportamiento de las operaciones aritméticas frente a valores normales y críticos (ceros, negativos), junto con técnicas de Caja Blanca aplicadas al método máximo. La efectividad de la suite se mide cuantitativamente con JaCoCo.

1.4 Criterios de Entrada (Entry Criteria)

Los criterios de entrada son las condiciones que deben cumplirse ANTES de empezar a ejecutar las pruebas. Ej: 'El código debe compilar sin errores'.

Criterios de Entrada

CE-01: El código fuente original de la clase Calculadora.java se encuentra integrado en la estructura de directorios estándar de Maven.

CE-02: El archivo de configuración pom.xml está disponible con las dependencias de JUnit 5 y JaCoCo correctamente declaradas.

CE-03: El entorno de desarrollo (IDE / Compilador) reconoce el proyecto y compila el código sin errores sintácticos.

1.5 Criterios de Salida (Exit Criteria)

Los criterios de salida son las condiciones que deben cumplirse para FINALIZAR las pruebas. Ej: 'El 80% de los tests deben pasar'.

Criterios de Salida

CS-01: Se ha ejecutado el 100% de los 15 casos de prueba unitarios planificados.

CS-02: El 100% de las pruebas unitarias finalizan con estado "Pasa" tras la corrección de los defectos hallados.

CS-03: Se alcanza una cobertura de líneas medida por JaCoCo superior al 80% en la clase bajo prueba.

1.6 Criterios de Suspensión

¿En qué condiciones pararáis las pruebas temporalmente? Ej: 'Si el código no compila, se suspenden las pruebas hasta que se corrija'.

Criterios de Suspensión

Las actividades de prueba se suspenderán temporalmente si se detecta un fallo de configuración en el entorno local (como la ausencia o desconfiguración del compilador JDK en la terminal) que impida compilar las clases de prueba (test-compile). Las pruebas se reanudarán una vez que se restablezcan las variables de entorno o se ejecuten a través del compilador interno del IDE.

1.7 Entorno de Pruebas

Sistema operativo	Windows 11
JDK	OpenJDK 17
IDE	IntelliJ IDEA
Herramienta de build	Maven 3.x
Framework de pruebas	JUnit 5.10
Herramienta de cobertura	JaCoCo 0.8.11
Repositorio	GitHub

1.8 Riesgos y Plan de Contingencia

Riesgo identificado	Probabilidad	Plan de contingencia
Conflictos locales con las herramientas de consola de Maven	Alta	Desviar la ejecución de los objetivos del ciclo de vida de Maven de forma directa al motor de ejecución gráfico de IntelliJ IDEA.

PARTE 2 – Especificación de Diseño de Pruebas (ISO 29119-3)

Esta sección documenta cómo habéis diseñado los casos de prueba: qué técnicas aplicasteis y por qué elegisteis esos casos concretos.

2.1 Características a probar

Para cada método o funcionalidad, explicad qué queréis verificar. Usad el criterio de PARTICIONES EQUIVALENTES: dividid los posibles valores de entrada en clases (válidos, inválidos, límite).

Método / Funcionalidad	Clase válida (entrada OK)	Clase inválida (entrada errónea)	Valores límite
<i>Ej: dividir(a, b)</i>	<i>a=10, b=2 → resultado=5</i>	<i>b=0 → excepción</i>	<i>a=0, b=1 → 0; a=MAX_DOUBLE</i>
<i>Sumar(a,b)</i>	<i>a=5, b=3 → 8</i>	<i>No aplica</i>	<i>a=10.0, b=0.0 → 10.0 (Neutro)</i>
<i>Restar(a,b)</i>	<i>a=10, b=4 → 6</i>	<i>No aplica</i>	<i>a=2, b=5 → -3 (Negativo)</i>
<i>Multiplicar(a,b)</i>	<i>a=-3, b=-3 → 9</i>	<i>No aplica</i>	<i>a=6, b=0 → 6 (Anulacion)</i>
<i>Maximo(a,b)</i>	<i>a=5, b=3 → 5</i>	<i>No aplica</i>	<i>a=5, b=5 → 5 (Identicos)</i>
<i>Porcentaje(a,b)</i>	<i>a=25, b=200 → 12.5</i>	<i>No aplica</i>	<i>T = 0 → 0 (Total cero)</i>

PARTE 3 – Especificación de Casos de Prueba (ISO 29119-3)

Documentad aquí cada caso de prueba. Debéis tener un mínimo de 15 casos. Añadid más copias de la tabla si necesitáis más espacio.

Para cada caso de prueba: (1) pensad en el nombre antes del código JUnit, (2) definid los datos de entrada y el resultado esperado, (3) luego implementadlo en JUnit 5 con ese mismo nombre.

ACLARACIONES: Las descripciones y todo lo demás va en el mismo orden, es decir, la primera línea de descripción va con la primera línea de técnica y así con todo. El estado previo a la prueba lo hemos eliminado ya que no afecta en nada.

CASOS DE PRUEBA TC-001/TC-002/TC-003	
Identificador	TC-001/TC-002/TC-003
Nombre / Descripción	Suma de positivos Suma de negativos Suma con cero
Clase / Método bajo prueba	Calculadora.sumar()
Técnica de diseño	Caja negra Caja negra Valores límite
Datos de entrada	a=5, b=3 a=-2, b=-4 a=10, b=0
Resultado esperado	8.5 -6 10
Resultado obtenido	8.5 -6 10
Estado (Pasa / Falla)	PASA PASA PASA
Observaciones / Defecto encontrado	Ninguno

CASO DE PRUEBA TC-004/TC-005	
Identificador	TC-004 / TC-005
Nombre / Descripción	Resta estandar Resta con resultado negativo
Clase / Método bajo prueba	Calculadora.restar()
Técnica de diseño	Caja negra Valores límite
Datos de entrada	$a=10, b=4$ $a=2, b=5$
Resultado esperado	6 -3
Resultado obtenido	6 -3
Estado (Pasa /Falla)	PASA PASA
Observaciones / Defecto encontrado	Ninguno

CASO DE PRUEBA TC-006/TC-007	
Identificador	TC-006 / TC-007
Nombre / Descripción	Multiplicacion por cero Multiplicacion de negativos
Clase / Método bajo prueba	Calculadora.multiplicar()
Técnica de diseño	Valores límite Caja negra
Datos de entrada	$a=6, b=0$ $a=-3, b=-3$
Resultado esperado	0 9
Resultado obtenido	0 9

Estado (Pasa /Falla)	<i>PASA</i> <i>PASA</i>
Observaciones / Defecto encontrado	<i>Ninguno</i>

CASO DE PRUEBA TC-008/TC-009	
Identificador	<i>TC-008 / TC-009</i>
Nombre / Descripción	<i>Division exacta estandar</i> <i>Division entre cero</i>
Clase / Método bajo prueba	<i>Calculadora.dividir()</i>
Técnica de diseño	<i>Caja negra</i> <i>Valores límite</i>
Datos de entrada	<i>a=10, b=2</i> <i>a=10, b=0</i>
Resultado esperado	<i>5</i> <i>Lanzar arithmeticException</i>
Resultado obtenido	<i>5</i> <i>Infinity (sin la excepcion)</i>
Estado (Pasa / Falla)	<i>PASA</i> <i>FALLA</i>
Observaciones / Defecto encontrado	<i>Ninguno</i> <i>El método no valida el divisor de forma segura.</i>

CASO DE PRUEBA TC-010/TC-011/TC-012**Identificador***TC-010 / TC-011 / TC-012***Nombre / Descripción***Máximo cuando $a > b$* *Máximo cuando $a < b$* *Máximo con valores idénticos***Clase / Método bajo prueba***Calculadora.maximo()***Técnica de diseño***Caja blanca**Caja blanca**Valores límite***Datos de entrada** *$a=8, b=3$* *$a=1, b=4$* *$a=5, b=5$* **Resultado esperado***8**4**5***Resultado obtenido***8**4**5***Estado (Pasa / Falla)***PASA*

	<i>PASA</i> <i>PASA</i>
Observaciones / Defecto encontrado	<i>Ninguno</i>

CASO DE PRUEBA TC-013/TC-014/TC-015	
Identificador	<i>TC-013 / TC-014 / TC-015</i>
Nombre / Descripción	<i>Porcentaje normal (25 de 200)</i> <i>Porcentaje equivalente (50 de 100)</i> <i>Porcentaje con total cero</i>
Clase / Método bajo prueba	<i>Calculadora.porcentaje()</i>
Técnica de diseño	<i>Caja negra</i> <i>Caja negra</i> <i>Valores límite</i>
Datos de entrada	<i>Valor = 25, total=200</i> <i>Valor = 50, total=100</i> <i>Valor = 10, total=0</i>
Resultado esperado	<i>12.5</i> <i>50</i> <i>0</i>
Resultado obtenido	<i>12.5</i> <i>50</i>

	0
Estado (Pasa / Falla)	PASA PASA PASA
Observaciones / Defecto encontrado	Ninguno

INFORME DE COBERTURA JACOCO Y JUNIT5

